

The Computation is a Graph: GNOMe for Extraction, Measurement, and Readout of Mechanistic Explicability

Sehaj Randhir Singh

Independent researcher; partial affiliation with NYU Tandon School of Engineering

Code, experiments, and paper source: github.com/sehajr-singhs/gnome

Abstract—Mechanistic interpretability asks what a trained network computes, yet most readouts answer with proxies—attention weights, saliency maps, activation patches—that are one step removed from the computation. We argue the computation already is a graph, and propose GNOMe (Graph Networks for Mechanistic Explicability): (1) *extract* a trained model’s circuit graph from blockwise mean-absolute Jacobians, so every edge is a real derivative (cost $\approx 1.4\%$ of training time); (2) *measure* explicability as ground-truth-free graph statistics (sparsity, modularity, effective depth, bottleneck, role diversity) plus *circuit recovery* where the true circuit is known by construction—trained boolean models recover their target circuits’ input-pair wiring with recall 1.00 in 9/9 models (F1 0.50–0.90), and the extracted graphs carry clean architectural signatures (MLP vs. transformer sparsity 0.71 vs. 0.43; role diversity 2.3 $\times$); (3) *read* explicability with graph networks that infer unit depth-roles from structure alone—no positional features—at 0.91–0.96 macro-F1 vs. 0.72–0.76 for a feature-only baseline, and that transfer to input sizes never seen during training while a baseline does not. A control shows the reading is architecture-specific (chance across architectures for every readout) and that semantic recovery is not predictable from structure by any method. Mechanistic explicability is a graph problem; GNOMe makes it measurable and learnable.

Index Terms—mechanistic interpretability, graph neural networks, circuit analysis, Jacobian attribution, explainable AI

I. INTRODUCTION

A trained network is a function implemented by wiring: which units read which inputs, how deep information flows, where bottlenecks form. Mechanistic interpretability tries to recover that wiring [1]–[3], but its instruments are proxies. Attention is a distribution over tokens, not a path through units [4]; saliency measures local sensitivity, not causal wiring [5]; patching-based circuit discovery infers edges by intervention, which is expensive and heuristic in thresholding [6]–[8].

The observation behind GNOMe: a model’s computation is a graph, and the weights already determine it. Each unit’s dependence on the previous layer is a Jacobian; averaged over inputs and thresholded, the blockwise Jacobians define a layered DAG—the *circuit graph*. Once the graph exists, explicability becomes graph statistics (measurable without ground truth), circuit recovery (certifiable where ground truth is known by construction), and a graph-network learning problem (readable, with generalization that a feature-only model cannot achieve, because roles and depths are multi-hop

quantities [9]–[11]). Because every claim is testable against known ground-truth circuits, GNOMe also delineates what graphs *cannot* read.

Contributions. (1) Blockwise-Jacobian circuit-graph extraction that is faithful, layered, and cheap. (2) A metric suite and composite Mechanistic Explicability Score (MES) with no ground-truth requirement, plus functional recovery against circuits known by construction. (3) A graph-network readout that infers unit depth-roles few-shot from structure alone and transfers across input sizes—with a controlled boundary: no readout transfers across architectures, and no readout predicts semantic recovery from structure. All numbers are produced by committed scripts on CPU and stored in committed JSONs.

II. METHOD

A. Extraction

A model must expose its *blocks* $u_k \mapsto u_{k+1}$ and unit dimensions. For each block we estimate the mean-absolute Jacobian

$$\mathbf{W}_k[i, j] = \mathbb{E}_x |\partial u_{k+1}[i] / \partial u_k[j]|, \quad (1)$$

exactly in closed form for linear blocks ($|\mathbf{W}|$ is input-independent), diagonally for elementwise blocks ($|f'(u_j)|$), and by one autograd pass per output unit otherwise, averaged over batches of 256. Per-layer relative thresholding ($rel_thresh \times$ layer-mean nonzero weight) yields a layered DAG; residual connections are folded into the block they close. Extraction is faithful (edges are derivatives) and cheap: mean 0.49 s per model vs. 34.4 s mean training ($\approx 1.4\%$).

B. Measurement

Five ground-truth-free metrics, each higher = more explicable: *sparsity* ($1 - |E| / \sum_k d_{k+1} d_k$), *modularity* (greedy Louvain on the undirected graph), *effective depth* (longest path), *bottleneck* ($1 - \min_k d_k / \sum_k d_k$), *role diversity* (fraction of distinct in/out-degree signatures). Where a ground truth exists, *recovery* compares input-pair co-wiring: does some hidden unit co-wire inputs $\{i, j\}$ the way some ground-truth gate does (precision/recall/F1 over input pairs). $MES = 0.4 F_1^{\text{rec}} + 0.2 \text{sp} + 0.2 \text{mod} + 0.2 \text{depth fidelity}$.

TABLE I
EXPLICABILITY METRICS AND RECOVERY (MEANS OVER 3 SEEDS FOR
BOOLEAN ROWS).

task	model	sparsity	depth	rec. F1
bool n7 g8	MLP	0.714	5.0	0.637
bool n7 g12	MLP	0.711	5.0	0.743
bool n7 g16	MLP	0.708	5.0	0.763
mod p5	MLP	0.722	5.0	—
mod p5	transf.	0.411	4.0	—
mod p7	MLP	0.715	5.0	—
mod p7	transf.	0.443	4.0	—

C. Readout

GCN [10] and GAT [11] stacks (three layers, width 64, pure PyTorch) predict node depth bins (input/early/mid/late/output) from structure-only features: normalized in/out degree and mean incoming/outgoing edge weight—no layer index. A feature-only MLP baseline sees identical features without edges. Training: Adam, 300 epochs, cross-entropy; few-shot protocol trains on K graphs and tests on held-out graphs.

D. Tasks

Boolean tasks are random layered DAGs of {AND, OR, XOR} gates over n_{in} bits (n_g gates, fixed seed); the ground-truth graph is the literal generated DAG. **Modular** tasks are $(a+b) \bmod p$ with one-hot inputs; the ground-truth circuit is the canonical Fourier/trigonometric construction [12], [13]. Models: 3-layer MLP ($d_{\text{in}} \rightarrow 32 \rightarrow 32 \rightarrow d_{\text{out}}$) and a one/two-layer transformer with explicit blocks; both trained to 100% train accuracy.

III. EXPERIMENTS

All numbers are from the committed results/benchmark_cpu.json and results/gnn_experiment.json; figures are regenerated by benchmarks/make_figures.py. Full wall time: ≈ 8 min on CPU.

A. Extraction is faithful: recovery

Across 9 trained boolean models (gate counts $8/12/16 \times 3$ seeds), every extracted graph contains the target circuit’s input-pair wiring: **recall 1.00 in 9/9 models**, mean F1 0.71 (range 0.50–0.90), mean precision 0.58 (0.33–0.81; Fig. 1). The only loss is precision—redundant co-wiring beyond the minimal circuit, the measurable cost of overparameterized learning. Recall never fails.

B. Measurement separates architectures

On identical modular-arithmetic tasks (Table I), MLPs and transformers produce measurably different circuit graphs: MLP sparsity 0.71–0.72 vs. transformer 0.41–0.44 (transformers wire $2\times$ denser graphs), role diversity 0.27 vs. 0.62 at $p=5$ ($2.3\times$), effective depth 5 vs. 4. Two architectures at identical test accuracy instantiate the same function with different wiring—visible in the graph.

TABLE II
FEW-SHOT DEPTH-ROLE READING (TEST MACRO-F1; STRUCTURE-ONLY
FEATURES).

K	GCN	GAT	baseline
2	0.871	0.868	0.727
4	0.913	0.887	0.720
6	0.891	0.869	0.729

TABLE III
TRANSFER OF STRUCTURAL READING (TRAIN: 6 MLP, 7-INPUT GRAPHS).

test set	GCN	GAT	baseline
same-size MLP (held out)	0.901	0.913	0.760
larger-size MLP ($n_{\text{in}}=9$, unseen)	0.957	0.934	0.755
transformer (control)	0.322	0.329	0.328

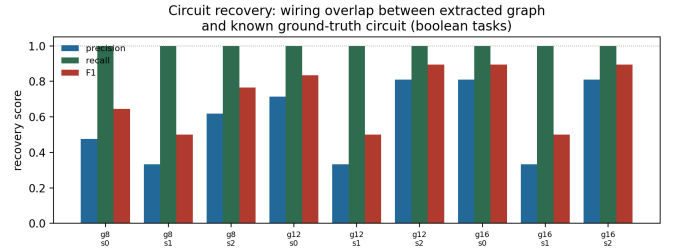


Fig. 1. Circuit recovery on boolean tasks: recall 1.00 in 9/9 models; F1 bounded only by redundant wiring.

C. Graph networks read explicability

Few-shot roles. Training on K graphs and testing on held-out graphs (Table II), graph readouts beat the feature-only baseline by 0.14–0.19 macro-F1 at every K ; at $K=4$: GCN 0.91, GAT 0.89, baseline 0.72. Depth is multi-hop; message passing computes it and flat features cannot.

Transfer: sizes yes, architectures no. Training on 7-input MLP circuit graphs and testing on 9-input graphs (an input count never seen during GNN training), the graph readouts transfer essentially losslessly (GCN 0.96, GAT 0.93—above same-size performance) while the baseline saturates (0.76); degree statistics shift with input count, structure does not (Table III). A control on transformer graphs of the same task collapses every readout—GCN, GAT, and baseline—to ≈ 0.32 (chance): depth-bin semantics and wiring statistics are architecture-specific. The reading is *size-invariant but architecture-specific*.

D. Boundary condition

Can a graph readout predict *recovery*—how much of an unseen target circuit a model learned—from structure alone? Collecting boolean graphs spanning input counts, gate counts, widths, and trained/untrained weights (recovery F1 0.40–0.95) and training on {5, 7}-input graphs to predict 9-input recovery F1, all readouts failed (test $R^2 < 0$ for GCN, GAT, and baseline: -4.1 , -5.9 , -1.8 ; committed in

results/recovery_control.json); the feature-only baseline’s failure shows the barrier is informational, not architectural: recovery is a statement about a target the extractor never sees. Readouts infer *structural* explicability and generalize it across sizes; *semantic* content requires comparing against the specification.

IV. CONCLUSION

GNOME converts mechanistic interpretability from a collection of heuristics into a measurable, learnable object: extract the computation as a graph from real derivatives, measure it with graph statistics and recovery against known circuits, and read it with graph networks that generalize across sizes. The framework’s testability pays off at the boundary: structural explicability is size-invariant but architecture-specific, and semantic recovery is not readable from structure by any method. The graph is not a metaphor for the computation—it is the computation. Scaling graph-based explicability readout to production-scale models, and designing cross-architecture transferable readers, are the immediate next steps.

REFERENCES

- [1] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, “Zoom in: An introduction to circuits,” *Distill*, 2020.
- [2] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah, “A mathematical framework for transformer circuits,” *Transformer Circuits Thread*, 2021.
- [3] J. Ferrando, G. Sarti, A. Bisazza, and M. R. Costa-jussà, “A primer on the inner workings of transformer-based language models,” *arXiv preprint arXiv:2405.00208*, 2024.
- [4] S. Jain, B. Voigt, and B. C. Wallace, “Attention is not explanation,” in *Proceedings of NAACL-HLT*, 2019.
- [5] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [6] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, “Interpretability in the wild: a circuit for indirect object identification in GPT-2 small,” *arXiv preprint arXiv:2211.00593*, 2022.
- [7] A. Conmy, A.-N. Măndoiu, A. Battaglia, N. Nanda, and S. Zohren, “Towards automated circuit discovery for mechanistic interpretability,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] F. Zhang and N. Nanda, “Towards best practices of activation patching in language models: Metrics and methods,” *arXiv preprint arXiv:2309.16042*, 2023.
- [9] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu, “Relational inductive biases, deep learning, and graph networks,” *arXiv preprint arXiv:1806.01261*, 2018.
- [10] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [12] N. Nanda and T. Lieberum, “A mechanistic interpretability analysis of addition,” *ARENA blog post*, 2022.
- [13] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, “Progress measures for grokking via mechanistic interpretability,” in *International Conference on Learning Representations (ICLR)*, 2023.